AI Tool (for all): Claude

Prompt:
(attached product_site/documentation/Homepage@1x.png)
here is the wireframe for a website for a school project, i want you to generate css variables for all the colors, plus a few extra i might need for other pages

Response:
Here are the CSS color variables for the CardWell wireframe, plus a few extras you'll likely need on other pages:

```
:root {

    /* ---------- Homepage (from mockup) ---------- */

    /* backgrounds */
    --color-bg: #ffffff;              /* main page background */
    --color-bg-alt: #f5f5f3;          /* gray sections (Why SRS, How it works) */
    --color-bg-dark: #1a1a1a;         /* bottom signup banner */
    --color-placeholder: #e8e8e6;     /* screenshot / image boxes */

    /* text */
    --color-text: #1a1a1a;            /* headings and main text */
    --color-text-muted: #555555;      /* paragraph / description text */
    --color-text-eyebrow: #666666;    /* small all-caps labels above headings */
    --color-text-inverse: #ffffff;    /* text on the dark banner */

    /* borders */
    --color-border: #e2e2e0;          /* card outlines, header divider */
    --color-border-strong: #1a1a1a;   /* highlighted CardWell comparison card */

    /* buttons */
    --color-btn-primary-bg: #1a1a1a;
    --color-btn-primary-text: #ffffff;
    --color-btn-primary-hover: #333333;
    --color-btn-secondary-bg: #ffffff;
    --color-btn-secondary-text: #1a1a1a;
    --color-btn-secondary-hover: #f0f0ee;

    /* ---------- Extras for other pages ---------- */

    /* links and focus */
    --color-link: #1a1a1a;
    --color-link-hover: #555555;
```

```
    --color-focus: #3b82f6;          /* outline when tabbing through inputs */

    /* form inputs (signup / login) */
    --color-input-bg: #ffffff;
    --color-input-border: #cfcfcc;
    --color-input-placeholder: #9a9a96;

    /* form feedback */
    --color-success: #2e7d32;
    --color-error: #c62828;
    --color-warning: #b26a00;
    --color-info: #1565c0;

    /* study page answer buttons */
    --color-again: #d84343;
    --color-hard: #e08a1e;
    --color-good: #3a9d5d;
    --color-easy: #3b82f6;

    /* pricing page */
    --color-badge-bg: #1a1a1a;       /* "MOST POPULAR" tag */
    --color-badge-text: #ffffff;

    /* misc */
    --color-shadow: rgba(0, 0, 0, 0.08);
    --color-overlay: rgba(0, 0, 0, 0.5);  /* behind modals / popups */
}
```

Usage looks like `background: var(--color-bg-alt);` or `border: 1px dashed var(--color-border);`. If you want a dark mode version too, I can add a `[data-theme="dark"]` block that remaps the semantic tokens.

Prompt:
what is the font used in the wireframe? can you give me a link so i can import?

Response:
I can't identify fonts from an image with certainty, but the wireframe looks most like **Ubuntu**. The headings have rounded, heavy letterforms, and the body text is light and open, which both match Ubuntu. **Fira Sans** is very similar, so if Ubuntu doesn't look quite right side by side, try that one. If you made the wireframe in a tool like Figma or Balsamiq, clicking any text layer will show the exact font name.

To import Ubuntu from Google Fonts, put this at the very top of your CSS file:

```
@import
url("https://fonts.googleapis.com/css2?family=Ubuntu:wght@400;500;700&display=swap");
```

Or put this in your HTML `<head>` instead, which loads a bit faster:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Ubuntu:wght@400;500;700&display=swap"
rel="stylesheet">
```

Then you can add it to your variables like the colors:

```
:root { --font-main: "Ubuntu", Arial, sans-serif; }
body { font-family: var(--font-main); }
```

That link loads weights 400 (regular), 500 (bold body text), and 700 (big headings). If you switch to Fira Sans, replace `Ubuntu` with `Fira+Sans` in the URL and `'Fira Sans'` in the variable.

Prompt:
(attached screenshot of study screen)
now i need the images for the homepage. generate the following:

a more zoomed in, more readable version of my study screen for the top of the homepage
(screenshot of study screen attatched)
avatars for the 4 fictional users near the bottom
2 pictures for the typical srs vs cardwell section, 1 showing 1 static card and the other showing 3 cards for the same word with different sentences
the others ill gather myself or take care of later

Response:
(study app picture)
Next, the “Typical SRS apps” image: one static card that always shows the exact same sentence.
(static card picture)
And the CardWell version: three stacked cards for the same word, each with a different real sentence that shows a different use:
(cardwell cards picture)
Finally, the four avatars, left to right: Elena (grad student), David (software engineer), Margaret (retired teacher), and Tomas (undergrad).
All four images are SVGs, so they stay sharp at any size, and you can download each one from its widget. The avatars all come in one file. If you’d rather have each avatar as its own file so you can drop it into its own `<img>` tag, I can split them up.

Prompt: make them pngs and split them up

Response:
I can’t create files directly in this chat, so this widget converts each image to a PNG in your browser. Every image gets its own download link, and if a link doesn’t work, you can right-click the image and choose “Save image as.”
(pngs with downloads)

The PNGs are exported at 2x resolution so they stay crisp on high-DPI screens. The study screen is 1280x1080 and the comparison cards are 1040x480. Each avatar is 500x500 with transparent corners outside the circle, so you don’t need `border-radius: 50%` in your CSS, though adding it won’t hurt.

Prompt:
how can i center my content in the middle of the screen so it isnt stretched at wide screen sizes? i would just set a max width on the body and center it, but that doesn't allow my bottom borders to stretch to the ends of the screen

Response:
Keep the sections full width, and use horizontal padding to squeeze the content into the middle. The backgrounds and borders still run edge to edge, but the content stops widening past your max width.

```
:root { --content-width: 1150px;
 --gutter: 40px;
 --page-padding: max(var(--gutter), calc((100% - var(--content-width)) / 2)); }
section, header, footer { padding-inline: var(--page-padding); }
section { border-bottom: 1px solid var(--color-border); }
```

Here's how the `max()` works. `calc((100% - 1150px) / 2)` takes whatever space is left beyond 1150px and splits it evenly between the left and right sides. On a 1920px screen that's `(1920 - 1150) / 2 = 385px` on each side, so the content is exactly 1150px wide and centered. On a 900px screen the calc would come out negative (`-125px`), so `max()` picks the static `40px` instead, and the content gets a comfortable gutter rather than touching the screen edges.

A percentage in padding is relative to the parent's width, so `100%` here means the full width of the body. That's better than `100vw`, which includes the scrollbar on Windows and can cause a small horizontal overflow.

For your layout, this means the dark email signup band, the gray "Why SRS" and "How it works" sections, and the nav divider all stretch across the full screen, while the text and cards inside them line up on the same 1150px column. To shrink the gutter on phones, override just that variable:

```
@media (max-width: 600px) { :root { --gutter: 20px; } }
```

Prompt:
in my homescreen, in the how it works section, im saving each step in an ordered list, with the li's containing a header for the top text and a div for the body text. i made it a grid with 3 columns so theyre side by side to match the formatting in my wireframe, and im trying to add divs for the circular numbers above but i can't get the circles looking good while keeping them in good positions and maintaining the format. is there another way i can do it?

Yes. Instead of adding extra divs, you can generate the circles with CSS counters and the `::before` pseudo-element. CSS numbers each `<li>` automatically and draws the circle as part of the list item, so it always sits right above the heading and never throws off your grid.

First, set up the counter on the list and the items:

```
ol {
list-style: none;
padding: 0;
counter-reset: step;
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 32px;
}
ol li { counter-increment: step; }
```

Then draw the circle:

```
ol li::before {
  content: counter(step);
  display: flex;
  align-items: center;
  justify-content: center;
  width: 40px;
  height: 40px;
  border: 1.5px solid var(--color-border-strong);
  border-radius: 50%;
  background: var(--color-bg);
  color: var(--color-text);
  font-weight: 700;
  margin-bottom: 16px;
}
```

Here's how it works. `counter-reset: step` on the `<ol>` creates a counter named `step` that starts at 0, and `counter-increment: step` on each `<li>` adds 1 to it. Inside `::before`, `content: counter(step)` prints the current value, so the items show 1, 2, and 3 without you

typing any numbers. `list-style: none` hides the browser's default numbers so they don't show up twice.

Making the pseudo-element `display: flex` turns it into a block-level box that sits on its own line above the heading. The `align-items` and `justify-content` properties center the number both vertically and horizontally inside the circle. The fixed width and height with `border-radius: 50%` keep it a perfect circle no matter what font you use, and `margin-bottom` creates the gap between the circle and the step title.

You can delete the number divs from your HTML now, so each `<li>` holds just the heading and the body text. If you add a fourth step later, it'll be numbered automatically.